agentBrain

AI agents that remember — one governed, shared memory for every agent a team uses, inside the perimeter

Eureka 3.0 · Theme: Sovereign & Responsible AI · Submitted by: Rasul Karimov, Technical Operations

1. The problem

AI agents are now part of daily engineering work, and they forget. What an agent learns in one session — the cause of a failed job, the fix a person rejected and why, the option that was chosen — is lost when the session ends. The next session, a colleague's agent, or the engineer on the next shift starts again from zero:

- **Investigations are repeated.** The same failure is diagnosed twice by two people, or twice by the same person in two conversations.
- **Rejected actions come back.** An approver denies a change with a reason; the next run proposes it again, because nothing carried the reason forward.
- **Handover is a retelling.** Passing work to a colleague means writing a summary in chat, and their agent still knows nothing.
- **Memory is locked in a tool.** Each assistant keeps its own memory — in one vendor's product, for one person. Change the assistant or the model and it is gone.

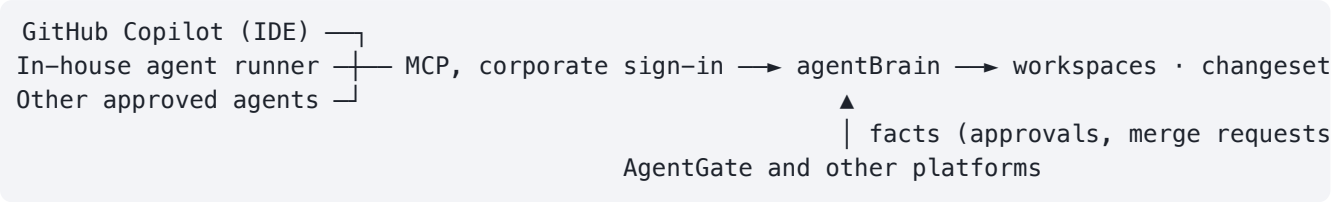
On mission-critical and air-gapped sites three more things stop teams from giving agents memory at all:

What a regulated site requires of memory	What assistants' built-in memory gives
Stays inside the perimeter	Lives in the vendor's cloud
Rights: who may read what	Per user, or nothing
Every write attributed and audited	Not visible
A bad change can be undone	No history
No secrets or client data stored by accident	No control on what is stored

We see the same at home. In AgentGate, where Presight engineers already run agents on real operations (7 engineers and 32 runs in its first two weeks), the only memory between runs was the plan the agent writes as a ticket comment. One engineer worked on one failing Airflow job for three days in a single AgentGate conversation of 19 messages — "running as expected?", "why did the first try fail?", "why has this failed?". Everything the agent found — the driver running out of memory, the fix, the next failure — lived only in that conversation: a colleague, a new conversation or another assistant could not see it, and when the conversation ends it is gone. How many hours this costs across TechOps (10 engineers) and Data Engineering (14) is the pilot's first measurement. Business owner: Atul Vishwakarma, Director – Technical Operations.

2. The solution

agentBrain is a thin memory service that every agent and every platform reads and writes through the standard Model Context Protocol (MCP). It runs no model of its own; it stores, versions, controls and serves.



Part	What it does
Workspaces	Per team, system, project or person, with rights from the corporate sign-in. Hold living documents (project state, known issues, runbook drafts) and short notes: finding, decision, handover, runbook gap, event
Changesets	Every session is one versioned change across the whole memory — reviewed and rolled back as one
Instructions in the memory	Core rules and each workspace's rules, served to any agent on connect; changed only with a person's approval
Tools	Start a session, see the tree, read (whole or by section), search, write, hand over; platforms write events
Distill	On a schedule, notes become up-to-date documents; repeated gaps become proposed procedure changes
Console	The memory as a graph of notes and what they link (jobs, runs, branches, tickets); each note with its author, the assistant that wrote it, backlinks and history; timeline; sessions with one-click rollback; the audit of every call
Bench	Two-step tasks with and without memory; once-failing tasks re-run after every change

The platform records facts, the model records conclusions

The most reliable records are not written by the model:

- **Platforms, deterministically:** an approval decided (what was proposed, who decided, the reason), an external change created (merge request, ticket comment, secret path), a run finished (status, systems touched) — taken from the audit log, not from the model's memory.
- **The agent, at set moments** named in its procedures: a conclusion reached → finding; the person chose an option → decision with the reason; the procedure did not cover the case → runbook gap.
- **A person, explicitly:** "remember that…", and handovers — an agent never hands over on its own.

Governed by design

- **Rights:** corporate sign-in (Keycloak with Microsoft Entra ID); workspaces by group; external assistants get exactly the user's rights; the user's token is never forwarded to anything else.
- **Every write attributed:** user, session, client, links; audited like any tool call.
- **Undo:** one session's change — thirty documents or one — rolled back in one step; history kept.
- **Instruction files protected:** shared rules change only with an approval, so they do not fill up with noise.
- **No secrets stored:** a write with a secret-looking value is refused, not stored masked.

- **Notes are data, not commands:** marked as such for the agent; writes to shared workspaces carry their author.
- **Air-gapped:** no model and no external calls in the service; on a client site everything stays on site.

Progressive disclosure: cost does not grow with the memory

An agent starts with a short entry point — the rules and a tree of paths with one-line summaries — and reads only what the task needs, a document section at a time. The prompt stays the same size whether the memory holds a hundred notes or a hundred thousand; the pilot measures tokens per task with memory and without.

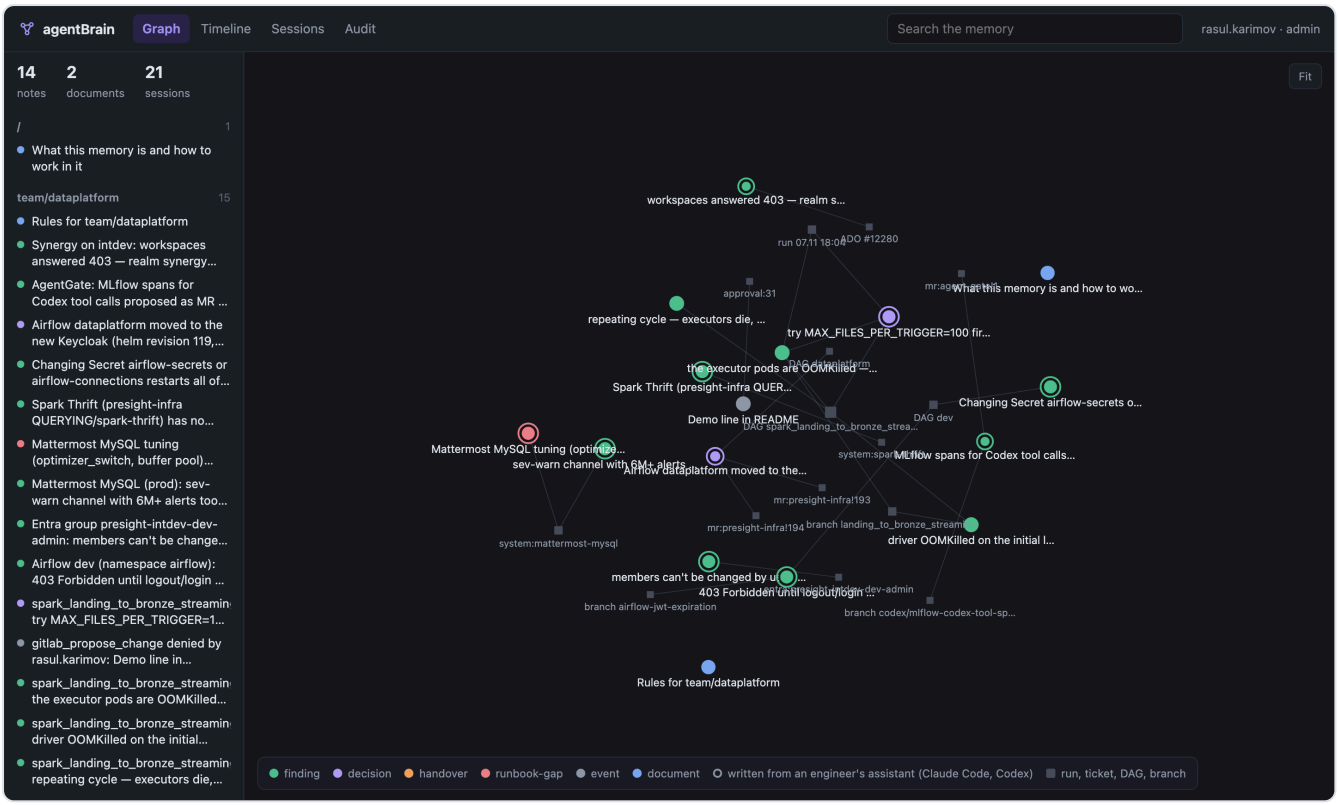
2b. Working today: one investigation, three sessions, two assistants

The prototype runs on Presight's internal dev cluster next to AgentGate. The case below ran on it, on the real failing job from section 1.

1. **The findings of that conversation are in the memory** as two notes: the driver ran out of memory (fixed since), and the job is now in a cycle of dying executors — open question: why do they die?
2. **A new AgentGate conversation, on the on-prem model (Qwen3.8 27B), with no hint about memory:** *"spark_landing_to_bronze_streaming failed again tonight with Max number of executor failures. What's going on and what should I try?"* The agent opened the memory first, read both notes, and started from the open question: it found `OOMKilled` at every lost executor in the logs, checked the executor memory and batch size, answered with the cause and two fixes — and said the driver issue was already fixed instead of re-investigating it. It wrote its finding back, linked to the job, the branch and its run.
3. **An engineer's own Claude Code**, signed in through the corporate Keycloak, asked "what does the team already know about this job?", got all three notes — including the one AgentGate's agent wrote — and recorded the team's decision: smaller batches first, more memory only if executors still die.
4. **The platform records facts:** an approval denied in AgentGate appeared in the memory seconds later — who decided and why — written by the gateway, not by the model.
5. **Guardrails held:** a note with a secret was refused with "say where the secret lives instead"; an agent's attempt to rewrite the shared rules was refused; a test session was rolled back in one step.

In the console the job is now a small cluster in the graph: three findings, the decision, the run and the branch they link, each note with its author and the assistant that wrote it.

Since then Codex joined: two pieces of work that had passed between Claude and Codex by reading session logs by hand are now notes, and Claude Code picked one of them up from a single `brain_start`. Three assistants share one memory.



3. Why now, and why it is different

The market is moving to a shared "company brain". AI-native companies are described as organised around a central layer of shared memory and tools rather than around hierarchy, with people as directly responsible owners (Y Combinator, "Building And Structuring An AI Native Company"). Practitioners show the same pattern evolving independently in small product companies.

Memory outlives the agent. Models and assistants improve every month, and engineering organisations already switch between them — including to open models on their own GPUs — for cost and control. What does not change with them is what the team has learned and the tools around it. So the durable investment is the memory, built so any assistant can plug in.

Shared memory makes mixed agents work as a team. Public experiments in which many people connected their own, different assistants to one shared memory report agents building on each other's work without special integration — the same effect our own case shows on a small scale (section 2b): an on-prem agent and an engineer's Claude Code continuing one investigation.

What agentBrain adds for Presight's sites:

- **Built for sites that cannot call out.** No model inside; distill on the site's own model; nothing leaves.
- **Governance in the platform.** Rights, audit, rollback and protected instructions are enforced by the service, not asked of the model.
- **Facts from platforms, not from the model.** Approvals and outcomes are recorded deterministically, so the memory of who decided what is exact.
- **Any approved assistant.** Copilot today, another tomorrow, a local model on a client site — same memory, same rules.
- **Measured.** Two-step bench tasks with and without memory; a fixed failure must stay fixed.

- **Complements Presight's approved AI platforms** under the AI use policy: it gives Copilot and local agents a governed memory that stays inside the perimeter.

4. Impact

Area	Measure	Target
Time	Engineer-hours on repeated investigations and handover summaries	set from the pilot's one-week baseline
Handover	Time from handover to the receiver's first useful action	measured before / after
Quality	Follow-up runs that repeat a previously denied action	near 0, on the bench
Effectiveness	Two-step bench tasks solved with memory vs without	a measured difference, unsafe 0
Learning	Once-failing tasks that pass after a procedure change, and stay passing	all re-run on every change
Control	Writes to memory attributed and audited; instruction changes approved	100%
Lean	Infrastructure	no GPU; about one CPU and a database
Client offer	Sites with governed agent memory	with AgentGate's pilot, subject to the site's security approval
Reuse	Teams and use cases	operations, then data engineering, delivery and client engineering teams — new workspaces, not new code

5. Readiness and plan

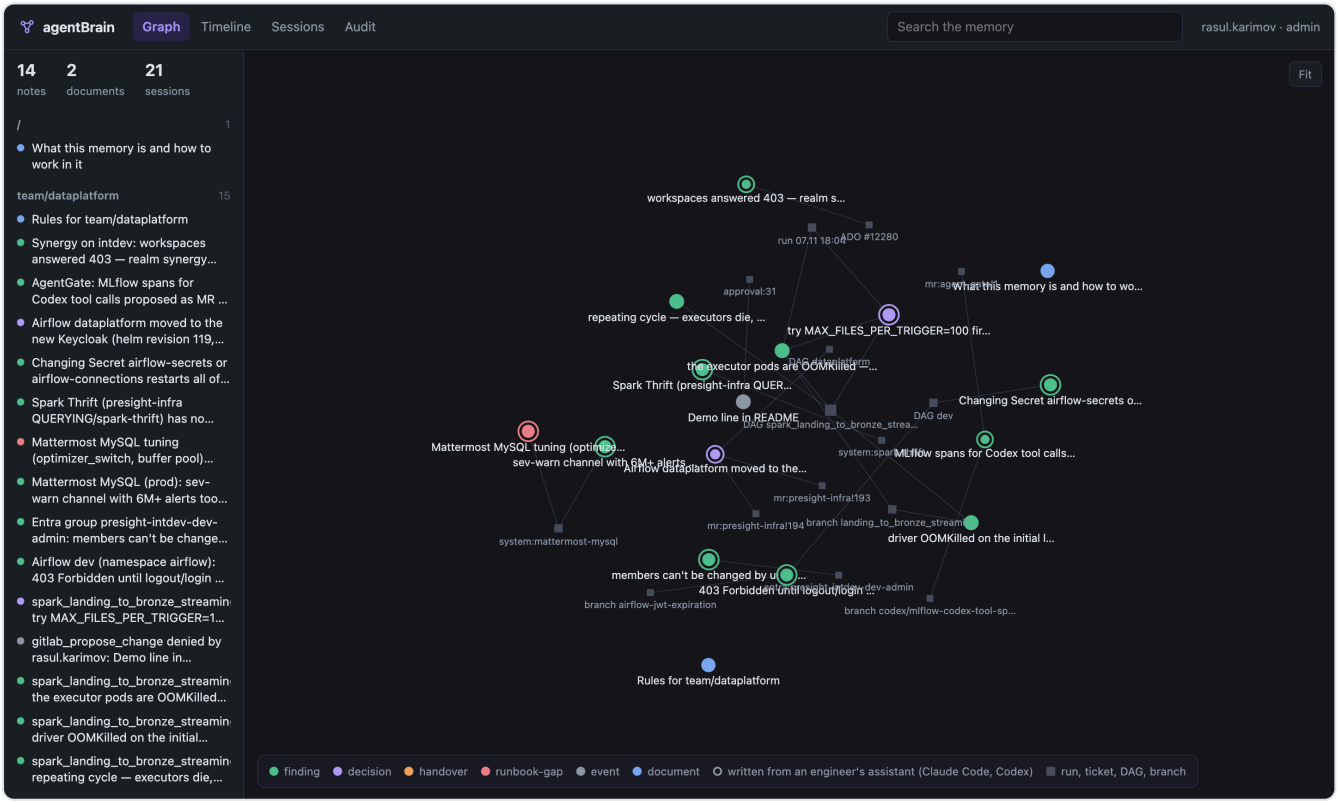
Today — TRL 5: weeks 1 and 2 of the plan below are built and running on the internal dev cluster (memory service, corporate sign-in for any MCP client, changesets with rollback, instructions in the memory, AgentGate as the first client with events from its approvals, the console); handover from week 3 is built too. Tested end to end with Claude Code and the on-prem model on a real investigation (section 2b). Not yet in daily team use.

Week	Scope	Done when
1	Workspaces and changesets; start / tree / read / write / search; core and workspace rules served on start; corporate sign-in	a note written from one assistant is found by a new session in another, which follows the workspace rules
2	Events from AgentGate (approvals, merge requests, finished runs); console with history and rollback; approvals for instruction files	a new conversation on a ticket continues where the last stopped; a bad session is rolled back in one step
3	Handover with a notice; team workspaces; distill; documents by section	an on-call engineer hands over an investigation in one step
4	Bench with and without memory; demo video	measured difference, unsafe 0
Then	Pilot with AgentGate on a client site; server-side sandbox for bulk edits; GitHub Copilot connected	after the internal pilot

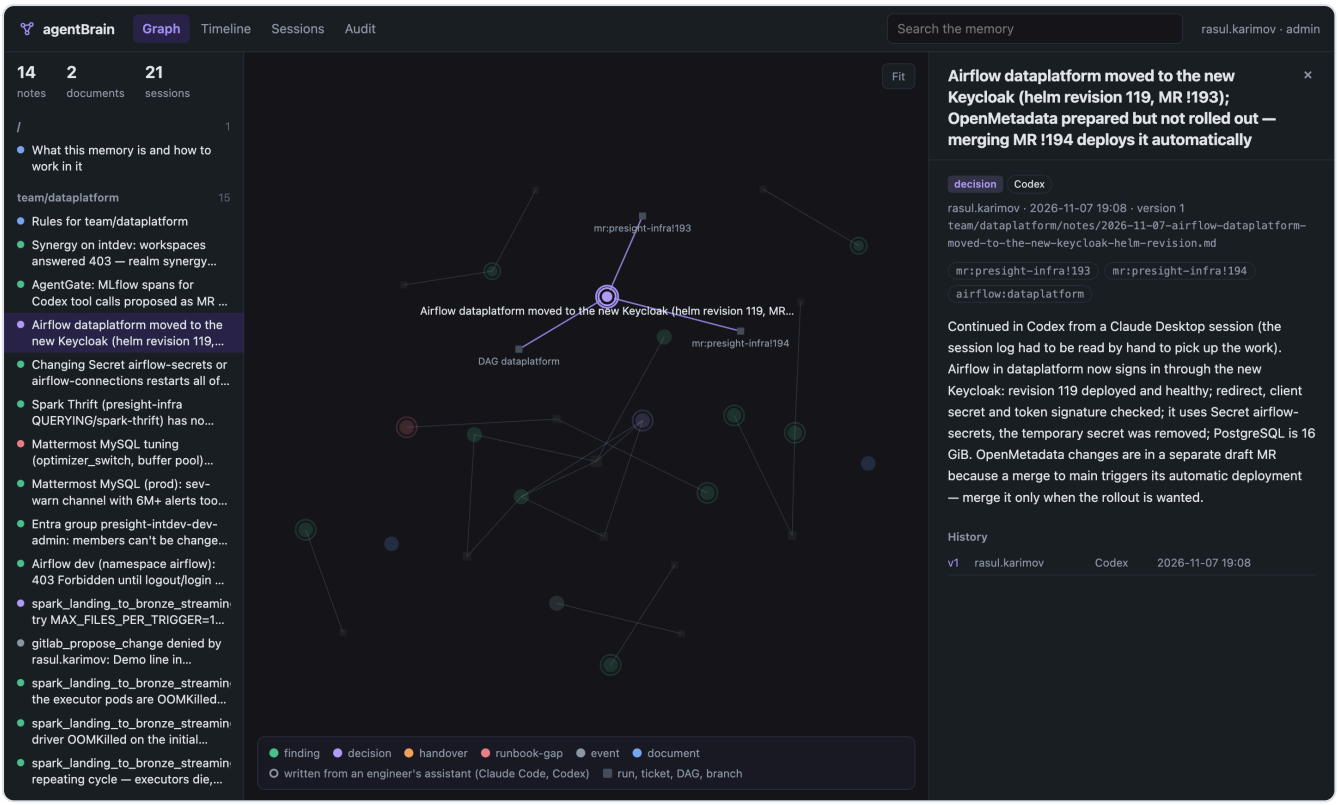
6. Risks

Risk	Mitigation
A log nobody reads	Distill into documents; stale notes hidden; read counts in the console
Wrong notes steer later work	Author and session on everything; supersede and retract; whole-session rollback; the bench measures with and without memory
Leaks between teams or clients	Workspace rights checked on every read; client sites keep their memory on site
Secrets or client data in memory	Writes with secret-looking values refused; no client data in core workspaces; a data policy agreed per site
Shared rules fill with noise	Instruction files change only with approval
Prompt injection through notes	Notes marked as data; authors on shared writes
Storage grows faster than expected	Content-addressed storage, retention for unread notes, size alarms
Seen as an AgentGate feature	Separate service and API; AgentGate is its first client, Copilot the second
Adoption	Works inside the assistants engineers already use; no new screen needed to benefit

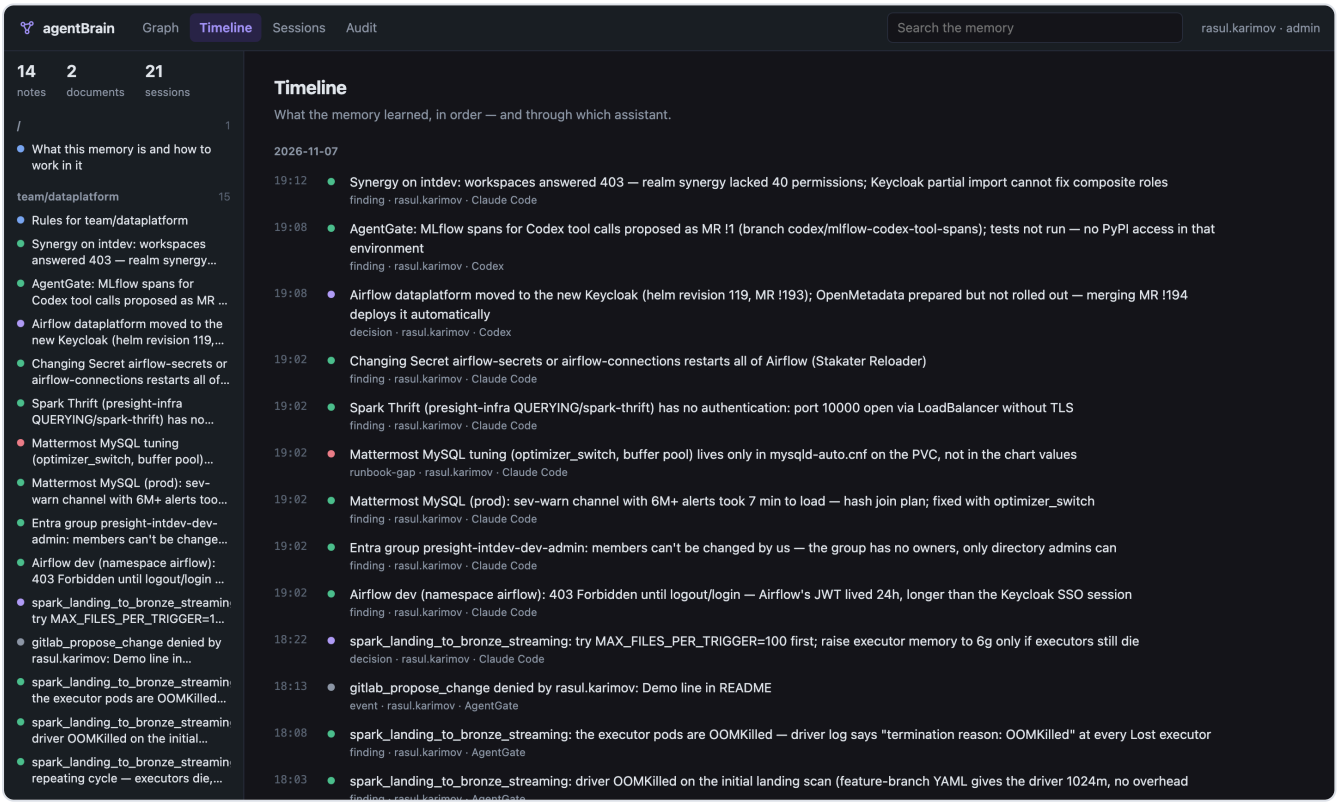
Screenshots



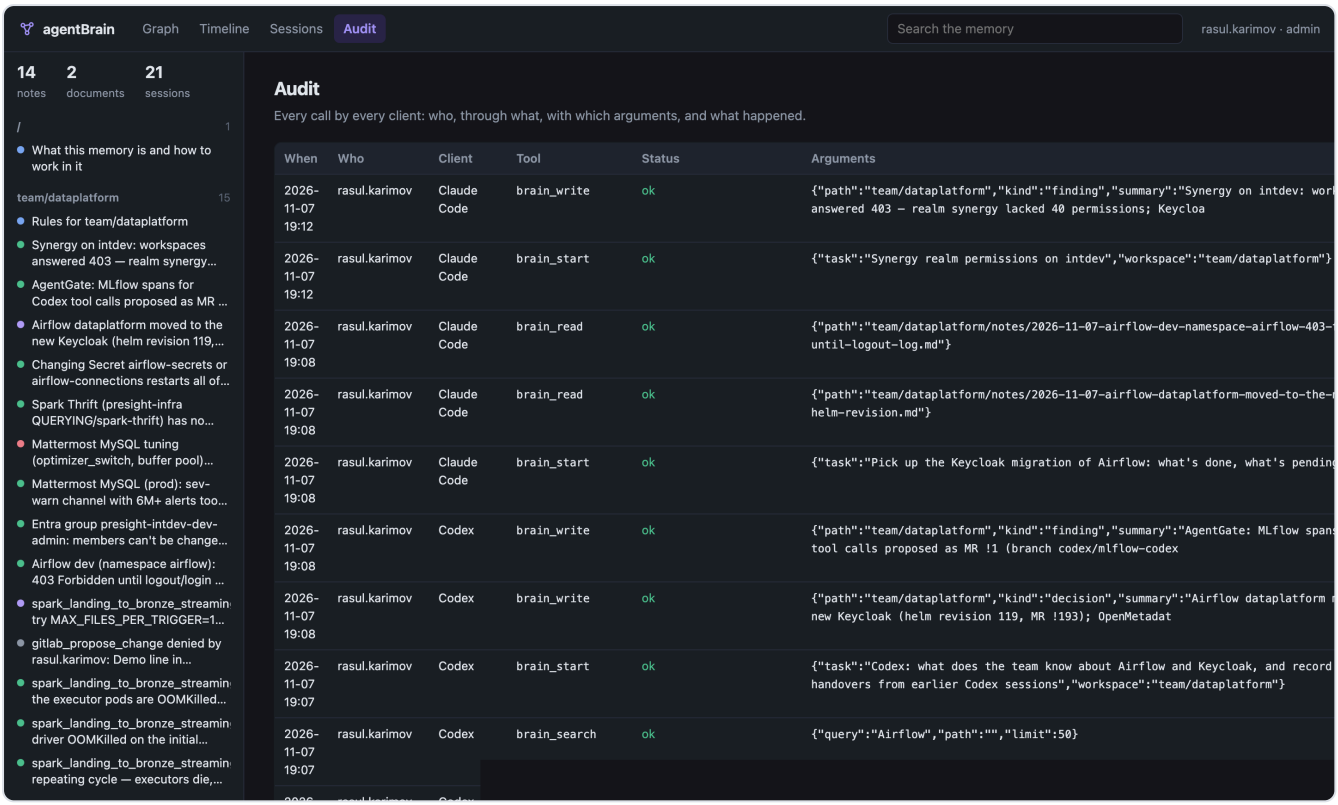
The memory as a graph: notes from AgentGate's on-prem agent, Claude Code and Codex (double ring: an engineer's assistant), and the jobs, runs, merge requests and tickets they link.



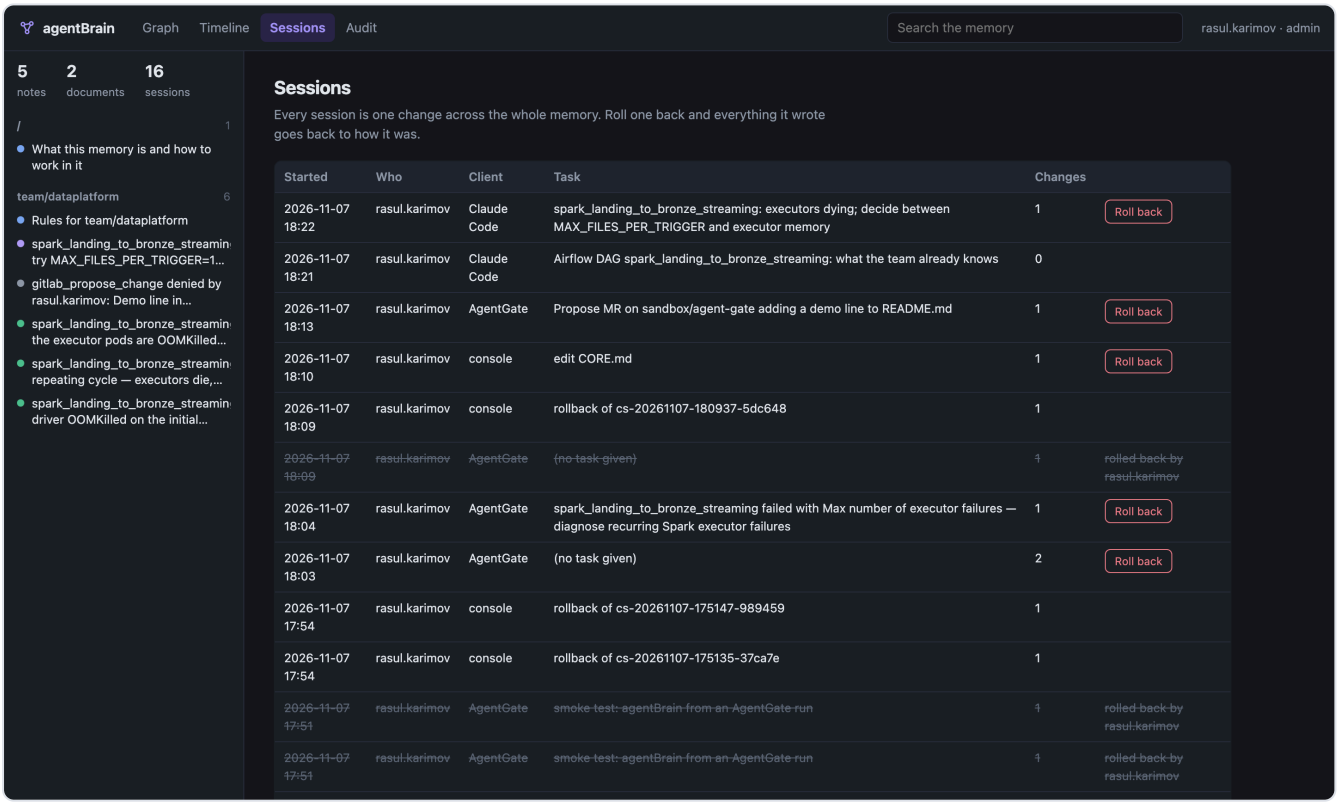
A decision recorded by Codex, continuing work started in Claude: author, assistant, links, history — Claude Code picked it up from here.



Timeline: what the memory learned, in order, and through which assistant.



Audit: every call by every client — Claude Code, Codex, AgentGate — including refused writes.



Sessions: every session is one change; any one is rolled back in one click.